stuffing method struggles with longer or more complex documents, while the refine method is more complex and may propagate errors if early summaries are of poor quality.

As we address the challenges that come from increased complexity, we also need to consider problems that may arise from integrating these new elements into our established app. In particular, we will encounter limitations related to performance and overall user experience. What happens if a user uploads a large document? Will our application be able to handle it without losing accuracy? Could the design of our processing pipelines lead to poor performance?

7.1.3 Building the document summarization web application

The summarization app kicks into gear when a user uploads either a PDF document or a Word document through the user interface. This action triggers a server-side process that handles the uploaded document. The server action uses LangChain.js to summarize the document. After summarizing the document, the server streams the summarized text back to the user in their preferred format—either as a paragraph or as bullet points—based on their prompt preferences. For simplicity, we will not keep the original document itself in context after processing.

The main challenges here are ensuring accurate summarization across various document types, managing context effectively during user interactions, and optimizing performance to provide timely responses during summarization tasks. Let's start with the main code that handles the document processing.

Project code: Summarization-examples

The code for this project is available at https://github.com/Generative-AI-Web-Apps/Code/tree/main/ch07/summarization-examples in the ch07/summarization-examples folder.

You'll also need either the OpenAI API or the Google API keys to access their REST API. For details on creating API keys, refer to the appendix:

```
// .env
OPENAI_API_KEY=<OPEN_AI_SECRET_KEY>
GEMINI_API_KEY=<GOOGLE_AI_SECRET_KEY>
```

NOTE The core LangChain.js library is implemented in TypeScript/JavaScript and does not require Python to be installed on your system. It runs natively in Node.js or browser environments. However, some community integrations or example projects may depend on Python tools or libraries and thus require a Python environment.

LOADING AND PREPROCESSING THE DOCUMENT

Ideally, we want the document chunks to contain relevant data ready to be used in an LLM prompt. Listing 7.1 shows a script that captures the core piece of this functionality.